Advanced Statistics - Exercise Sheet 1

Daniele Ghezzi - 2195461
Physics of Data - University of Padua

March 23, 2026

1 Exercise 1: Vectors and Data Frames

The table (from `https://en.wikipedia.org/wiki/List_of_lochs_of_Scotland`*) reports the volume, area, length, and the maximum and mean depth of several Scottish lakes.*

1.1

Create R vectors containing the lake names and the corresponding numerical variables. Build a data frame called scottish.lakes.

```r
# We define the vectors corresponding to the table's columns and then the dataframe.
names = c("Loch Ness", "Loch Lomond", "Loch Morar", "Loch Tay", "Loch Awe", "Loch Maree", "Loch Ericht", "Loch Lochy", "Loch Rannoch", "Loch Shiel", "Loch Katrine", "Loch Arkaig", "Loch Shin")
volumes = c(7.45, 2.6, 2.3, 1.6, 1.2, 1.09, 1.08, 1.07, 0.97, 0.79, 0.77, 0.75, 0.35)
areas = c(56, 71, 27, 26.4, 39, 28.6, 18.6, 16, 19, 19.5, 12.4, 16, 22.5)
lengths = c(39, 36, 18.8, 23, 41, 20, 23, 16, 15.7, 28, 12.9, 19.3, 27.8)
max_depths = c(230, 190, 310, 150, 94, 114, 156, 162, 134, 128, 151, 109, 49)
mean_depths = c(132, 37, 87, 60.6, 32, 38, 57.6, 70, 51, 40, 43.4, 46.5, 15.5)

scottish.lakes <- data.frame(names, volumes, areas, lengths, max_depths, mean_depths)
scottish.lakes
```

```
          names volumes areas lengths max_depths mean_depths
1     Loch Ness    7.45  56.0    39.0        230       132.0
2   Loch Lomond    2.60  71.0    36.0        190        37.0
3    Loch Morar    2.30  27.0    18.8        310        87.0
4      Loch Tay    1.60  26.4    23.0        150        60.6
5      Loch Awe    1.20  39.0    41.0         94        32.0
6    Loch Maree    1.09  28.6    20.0        114        38.0
7   Loch Ericht    1.08  18.6    23.0        156        57.6
8    Loch Lochy    1.07  16.0    16.0        162        70.0
9  Loch Rannoch    0.97  19.0    15.7        134        51.0
10   Loch Shiel    0.79  19.5    28.0        128        40.0
11 Loch Katrine    0.77  12.4    12.9        151        43.4
12  Loch Arkaig    0.75  16.0    19.3        109        46.5
13    Loch Shin    0.35  22.5    27.8         49        15.5
```

1.2

Determine the lake with the largest and the smallest volume, and the lake with the largest and the smallest surface area.

```r
# We determine the lake with largest/smallest volume/surface using
# which.max and which.min that select the index of maximum/minimum value of a vector
# Then we get the lake's name by accessing the df attribute.
largest_volume <- scottish.lakes$names[which.max(scottish.lakes$volumes)]
largest_volume
smallest_volume <- scottish.lakes$names[which.min(scottish.lakes$volumes)]
smallest_volume

largest_surface <- scottish.lakes$names[which.max(scottish.lakes$areas)]
largest_surface
smallest_surface <- scottish.lakes$names[which.min(scottish.lakes$areas)]
smallest_surface
```

```
[1] "Loch Ness"
[1] "Loch Shin"
[1] "Loch Lomond"
[1] "Loch Katrine"
```

1.3

Order the data frame with respect to the lake area and determine the two lakes with the largest area.

```r
# We sort the dataframe by using order, which sorts the selected vector and returns
# the element indexes. Then, since the elements are in ascending order, we tail the last two
# elements to get the two largest lakes.
scottish.lakes_sorted <- scottish.lakes[order(scottish.lakes$areas), ]
scottish.lakes_sorted
tail(scottish.lakes_sorted, 2)$names
```

```
[1] "Loch Ness"   "Loch Lomond"
```

1.4

By summing all the lake areas, compute the total area covered by these lakes.

```r
# The total area is simply the sum of the areas.
sum(scottish.lakes$areas)
```

```
[1] 372
```

2 Exercise 2: 100m Athletics Records

The website

`https://www.alltime-athletics.com/m_100ok.htm`

contains a list of the best recorded times in official men's 100 m races.

2.1

Open the results on the Web, reported as a table and create a tibble that contains all the results.

```r
# Initial attempt to load the data directly.
# This fails to format correctly because the source is an HTML page, not a tabular file.
library(tibble)

url <- 'https://www.alltime-athletics.com/m_100ok.htm'
data <- read.csv(url, header=F)
as_tibble(data)
```

A direct CSV import was attempted, but due to the HTML structure of the source the import failed. We can see it through the Terminal:

```
# A tibble: 5,057 × 1
   V1
   <chr>
 1 <HTML>
 2 <HEAD>
 3 <META Name=description Content=100m, 100 meter, 100 metres, statistics>
 4 <META Name=keywords Content=friidrott, friidrottsstatistik, track, field, at...
 5 <TITLE>Men's 100m</TITLE>
 6 </HEAD>
 7 <BODY BGCOLOR=White LINK=Green VLINK=Red ALINK=Blue>
 8 <center><font face=Arial size=2><b><a href=http://www.alltime-athletics.com/...
 9 <img src=../pic/athletics50.jpg width=295 height=53 border=0 alt=Goto -> Tra...
10 <b> updated and maintained by<br><strong><A class=sublink href=javascript:ta...
```

2.2

To parse the Web data, use the ***rvest*** *and* ***tidyverse*** *packages and the suggested code.*

```r
# Correct web approach. We extract the text inside the HTML <pre> tags
# using 'rvest' and parse it as a fixed width file to perfectly capture the columns.
library(rvest)
library(tidyverse)
men100m_html <- read_html("http://www.alltime-athletics.com/m_100ok.htm")
men100m_html |> html_nodes(xpath = "//pre") |> html_text() -> men100m_list
men100m_tbl <- read_fwf(men100m_list)
```

We utilized the `rvest` package to scrape the specific `<pre>` node containing the fixed-width text. The `read_fwf` function was necessary because the data is aligned by character position rather than separated by commas or tabs.

2.3

The resulting tibble has all data represented as "character". Manipulate the tibble to convert the columns to the proper format.

We removed unnecessary bottom rows, parsed the `Wind` column by removing the "±" symbol[1], and converted the numerical data in integer/numeric and the dates into a proper `YYYY-MM-DD` format.

```r
# Data cleaning pipeline: we drop irrelevant bottom rows, convert text into
# proper numeric/date formats, and rename variables for clarity.
men100m_tbl <- men100m_tbl |>
    slice(1:4848) |>
    mutate(X1 = as.integer(X1)) |>
    mutate(X2 = as.numeric(str_remove(X2, "A"))) |>
    mutate(X3 = as.numeric(str_remove(X3, "+-"))) |>
    mutate(X9 = as.Date(X9, format = "%d.%m.%Y")) |>
    rename(
        Position = X1,
        Time = X2,
        Wind = X3,
        Athlete = X4,
        Nation = X5,
        Birth_Date = X6,
        Result_place = X7,
```

[1]Due to Latex formatting, in the code here there is $+-$ while in the R code we have $\pm$.

```r
        Place = X8,
        Race_Date = X9
  )
print(men100m_tbl, n=10)
# NA values appear when there is no data
```

```
# A tibble: 4,848 × 9
   Position  Time  Wind Athlete  Nation Birth_Date Result_place Place Race_Date
      <int> <dbl> <dbl> <chr>    <chr>  <chr>      <chr>        <chr> <date>
 1        1  9.58   0.9 Usain B... JAM    21.08.86   1            Berl... 2009-08-16
 2        2  9.63   1.5 Usain B... JAM    21.08.86   1            Lond... 2012-08-05
 3        3  9.69   0   Usain B... JAM    21.08.86   1            Beij... 2008-08-16
 4        3  9.69   2   Tyson G... USA    09.08.82   1            Shan... 2009-09-20
 5        3  9.69  -0.1 Yohan B... JAM    26.12.89   1            Laus... 2012-08-23
 6        6  9.71   0.9 Tyson G... USA    09.08.82   2            Berl... 2009-08-16
 7        7  9.72   1.7 Usain B... JAM    21.08.86   1rA          New ... 2008-05-31
 8        7  9.72   0.2 Asafa P... JAM    23.11.82   1rA          Laus... 2008-09-02
 9        9  9.74   1.7 Asafa P... JAM    23.11.82   1h2          Rieti 2007-09-09
10        9  9.74   0.9 Justin ... USA    10.02.82   1            Ad-D... 2015-05-15
```

2.4

Once `men100m_tbl` *is properly formatted, the following plots are realized:*

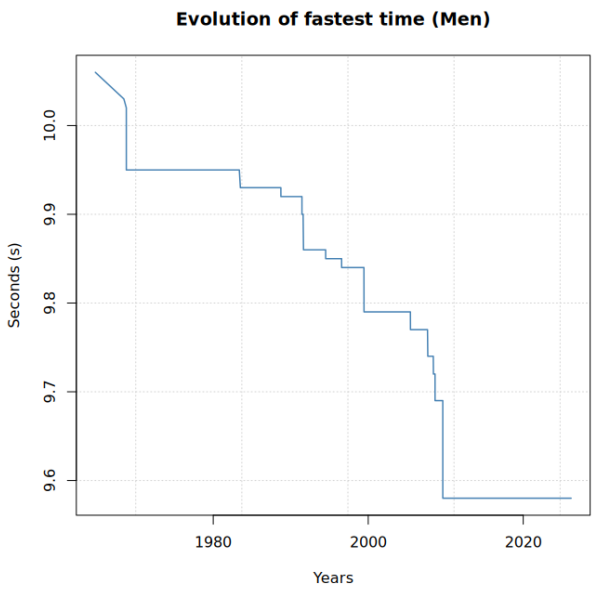
1. *evolution of the fastest time as a function of date of the races;*
2. *which country had the largest number of the fastest runners each year?*

To track the world record progression, we sorted the dataset chronologically and applied a cumulative minimum function to the `Time` column.

```r
# a) Calculate the cumulative minimum of the times to track the historical
# world record progression, then plot it as a line graph.
df_plot1 <- men100m_tbl |>
  arrange(Race_Date) |>
  mutate(AbsRecord = cummin(Time))

print(df_plot1[, c("Race_Date", "AbsRecord")], n=10)

plot(df_plot1$Race_Date, df_plot1$AbsRecord,
     type = "l",                    # "l" stands for line
     col = "steelblue",
     lwd = 1.5,                     # line width
     main = "Evolution of fastest time (Men)",
     xlab = "Years",
     ylab = "Seconds (s)",
     panel.first = grid(col = "lightgray") # grid
)
```

```
# A tibble: 4,848 x 2
   Race_Date  AbsRecord
   <date>         <dbl>
 1 1964-10-15     10.1
 2 1968-06-20     10.0
 3 1968-10-13     10.0
 4 1968-10-14      9.95
 5 1968-10-14      9.95
```

```
 6 1968-10-14       9.95
 7 1968-10-14       9.95
 8 1972-08-31       9.95
 9 1975-08-20       9.95
10 1975-08-20       9.95
```



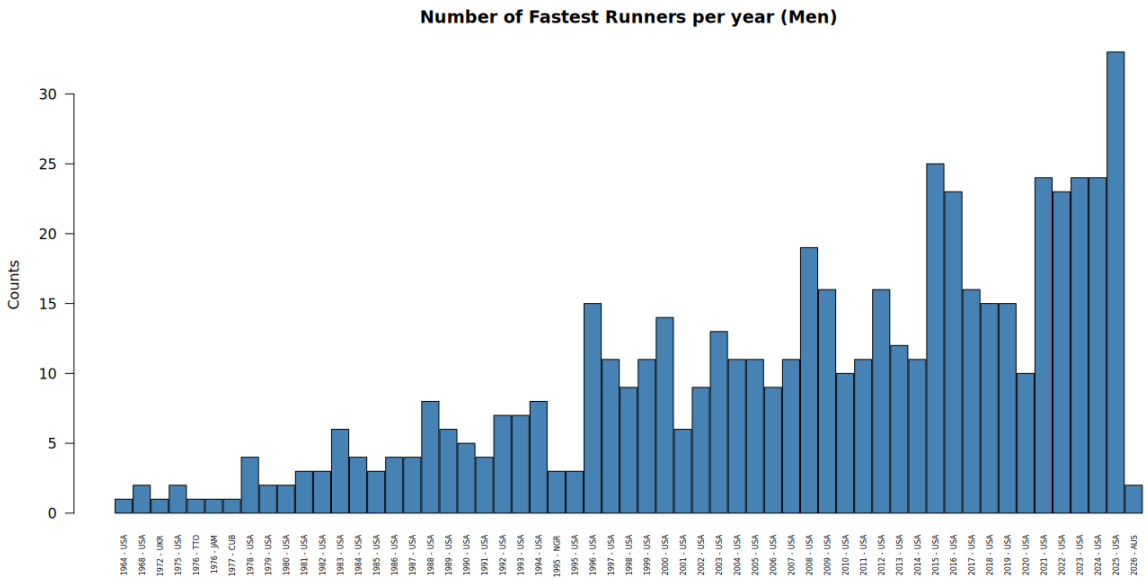
In the plot we have the world record over time. The use of `cummin()` ensures that the value only decreases when a new, faster time is recorded in the dataset.
For the second request, we utilize the `.by` argument within the `summarise()` function to group the data by both year and nation. The `n_distinct()` function is then applied to count how many unique athletes appeared at least once per year for each country. Finally, we use the `slice_max()` function to select the maximum number of athletes for each year, thereby identifying the nation with the highest count of fastest runners for every year recorded.

```r
# b) Identify the dominant nation per year by counting unique athletes,
# and visualize the result using a dense, customized barplot.
top_nations_by_year <- men100m_tbl |>
    drop_na(Race_Date) |>
    # Write each date with only the year
    mutate(Year = format(Race_Date, "%Y")) |>
    # .by groups by year and nation, and we create the new dataframe by adding
    # the number of athlete. We use n_dinstinct as an athlete can be present multiple times in a year
    summarise(Unique_Runners = n_distinct(Athlete), .by = c(Year, Nation)) |>
    # for each year (column n=1) we select the maximum with respect to the number of unique runners
    slice_max(Unique_Runners, n = 1, by = Year) |>
    # we sort
    arrange(Year) |>
    # for plotting reason, we merge the year column with nation one in a new column
    mutate(Year_Country = paste(Year, Nation, sep = " - "))
top_nations_by_year

barplot(
    height = top_nations_by_year$Unique_Runners,
    names.arg = top_nations_by_year$Year_Country,
    col = "steelblue",
    main = "Number of Fastest Runners per year (Men)",
    xlab = "",
```

```
    ylab = "Counts",
    border = "black",
    space = 0.05,
    las = 2,                                        # rotate by 90 degrees
    cex.names = 0.5                                # smaller the label text
)
```

```
# A tibble: 57 × 4
   Year  Nation Unique_Runners Year_Country
   <chr> <chr>           <int> <chr>
 1 1964  USA                 1 1964 - USA
 2 1968  USA                 2 1968 - USA
 3 1972  UKR                 1 1972 - UKR
 4 1975  USA                 2 1975 - USA
 5 1976  TTO                 1 1976 - TTO
 6 1976  JAM                 1 1976 - JAM
 7 1977  CUB                 1 1977 - CUB
 8 1978  USA                 4 1978 - USA
 9 1979  USA                 2 1979 - USA
10 1980  USA                 2 1980 - USA
```

In the barplot we can observe that the highest number of fastest athletes has consistently belonged to the USA since 1995.



2.5

Repeat the same analysis for the women's 100 m races using

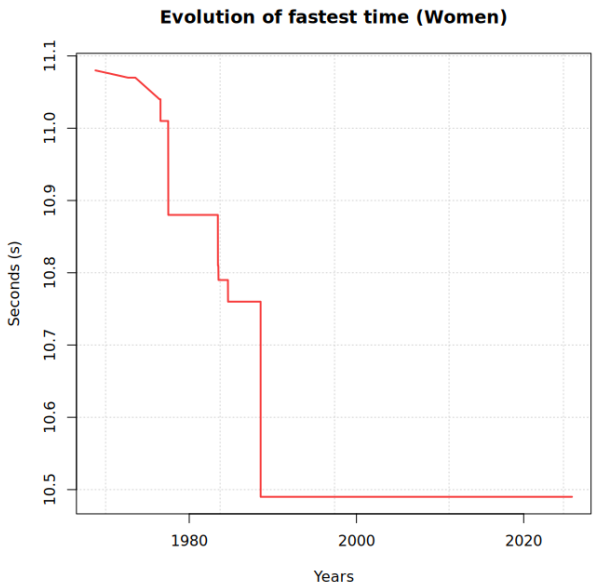
https://www.alltime-athletics.com/w_100ok.htm

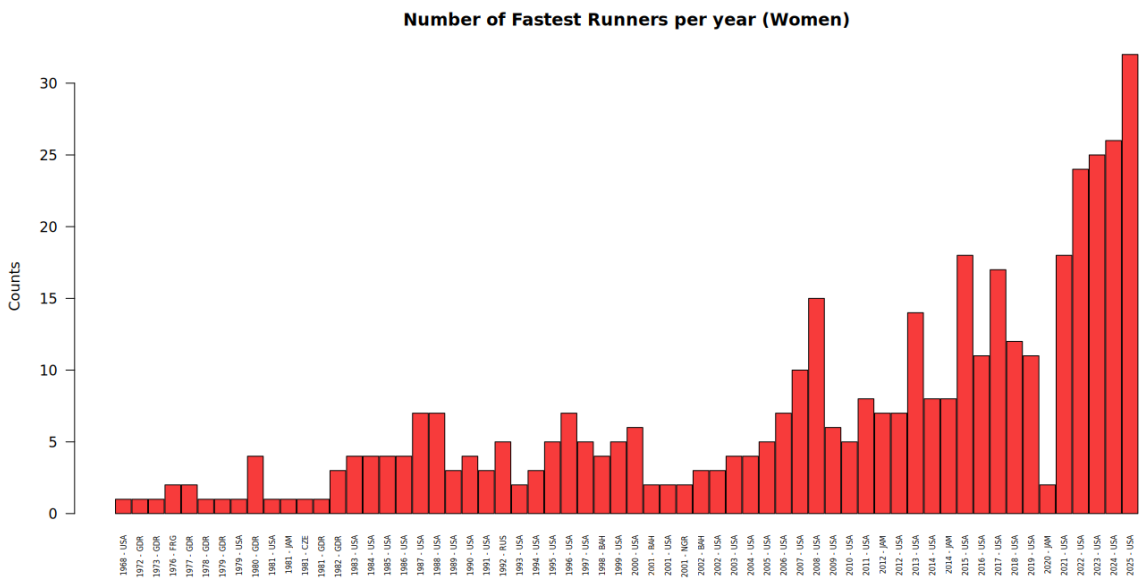
and compare the results between men and women.

We repeat the analysis with women's dataset. For the sake of brevity, terminal outputs are not reported, as the processing steps follow the same logic as the men's analysis.

```
# Same analysis with women dataset
url2 <- 'https://www.alltime-athletics.com/w_100ok.htm'

```

```r
women100m_html <- read_html(url2)
women100m_html |> html_nodes(xpath = "//pre") |> html_text() -> women100m_list
women100m_tbl <- read_fwf(women100m_list)

women100m_tbl <- women100m_tbl |>
    slice(1:4848) |>
    mutate(X1 = as.integer(X1)) |>
    mutate(X2 = as.numeric(str_remove(X2, "A"))) |>
    mutate(X3 = as.numeric(str_remove(X3, "+-"))) |>
    mutate(X9 = as.Date(X9, format = "%d.%m.%Y")) |>
    rename(
        Position = X1,
        Time = X2,
        Wind = X3,
        Athlete = X4,
        Nation = X5,
        Birth_Date = X6,
        Result_place = X7,
        Place = X8,
        Race_Date = X9
  )
print(women100m_tbl, n=10)

df_plot1 <- women100m_tbl |>
  arrange(Race_Date) |>
  mutate(AbsRecord = cummin(Time))
print(df_plot1[, c("Race_Date", "AbsRecord")], n=10)

plot(df_plot1$Race_Date, df_plot1$AbsRecord,
     type = "l",
     col = "#f73b3b",
     lwd = 2,
     main = "Evolution of fastest time (Women)",
     xlab = "Years",
     ylab = "Seconds (s)",
     panel.first = grid(col = "lightgray")
)

top_nations_by_year <- women100m_tbl |>
    drop_na(Race_Date) |>
    mutate(Year = format(Race_Date, "%Y")) |>
    summarise(Unique_Runners = n_distinct(Athlete), .by = c(Year, Nation)) |>
    slice_max(Unique_Runners, n = 1, by = Year) |>
    arrange(Year) |>
    mutate(Year_Country = paste(Year, Nation, sep = " - "))
top_nations_by_year

barplot(
    height = top_nations_by_year$Unique_Runners,
    names.arg = top_nations_by_year$Year_Country,
    col = "#f73b3b",
    main = "Number of Fastest Runners per year (Women)",
    xlab = "",
    ylab = "Counts",
    border = "black",
    space = 0.05,
    las = 2,
    cex.names = 0.5
)
```





From the first plot, we observe that the decreasing trend in record times for women mirrors the one observed for men, although characterized by higher values. The second plot reveals a similar pattern to the men's analysis: the highest number of fastest runners per year has consistently belonged to the USA since 2002, confirming their leadership in both categories.